

From Fixed Group Round-Robin to Guarded Adaptivity in the Dining Philosophers Problem: Formal Bounds, Native Multi-Architecture Validation, and Coordinator Removal

Aashish BishowKarma

Abstract—The Dining Philosophers Problem (DPP) is a compact model of deadlock, starvation, fairness, and resource contention. Lee (2023) already described a group round-robin (GRR) construction that rotates $\lfloor N/2 \rfloor$ nonadjacent philosophers, so fixed GRR is treated as prior art. We formalize its guarantees, use Age-MWIS as an aggressive adaptive reference, and study Guarded-Fill GRR (GF-GRR), which preserves the rotating service guard while filling otherwise idle conflict-free capacity. In a synchronous simulator, GF-GRR recovered 79.3–99.7% of the throughput improvement of Age-MWIS over GRR across 12 core conditions while retaining the GRR service guard; stress tests reached $N = 10,001$ and selector microbenchmarks $N = 10^6$.

A pre-submission audit invalidated an earlier native-thread dataset because of a recursive sleep-payload bug and a drain-path payload mismatch. A subsequent accounting audit identified two further semantic issues: the reported “throughput” counted requests by arrival cohort rather than completions inside the measurement interval, and the coordinator-free DGuard epoch was frozen during warmup. We corrected all four issues before freezing the native evidence. The final C++17 benchmark counts an operation only when it completes inside a 2.0 s measurement window, starts the DGuard epoch before worker release, passes `-O0 ASan/UBSan` smoke tests, and is then compiled with `-O3`. The final matrix contains 960 accepted runs on native x86₆₄ and ARM64 hardware: $N \in \{101, 501\}$, three payloads (sleep, ledger, KV), saturated and bursty demand, five policies, and eight paired seeds.

At saturated $N = 501$ sleep, GF-GRR delivered 60.54k/s on x86₆₄ and 115.92k/s on ARM64 with p99 waits of 17.74 and 17.05 ms. DGuard was about 30–32% slower and had p99 waits of 78.54 and 39.05 ms. Age-MWIS was statistically indistinguishable from GF-GRR in x86₆₄ throughput and 7.3% faster on ARM64, but its p99 wait rose to 204.79 and 26.04 ms and its Jain fairness was lower on both platforms. For ledger and KV, coordinator removal is architecture dependent: on ARM64 DGuard increases saturated throughput by 48.5–55.2% while worsening p99 and fairness, whereas on x86₆₄ its throughput is near-tied on ledger and slightly below GF-GRR on KV, with neither throughput contrast significant after Holm correction, while the p99 and fairness penalties remain significant. Thus the final multi-architecture evidence supports a workload- and platform-dependent frontier rather than a universal centralization/decentralization ranking.

Index Terms—Dining philosophers, deadlock, starvation, bounded waiting, round robin, independent set, conflict graph, scheduling, concurrency, fairness.

I. INTRODUCTION

Dijkstra’s Dining Philosophers Problem (DPP) is one of the canonical abstractions for concurrent resource allocation. Philosophers alternate between thinking and eating, each

philosopher needs the two forks shared with its immediate neighbors, and therefore adjacent philosophers cannot eat simultaneously [1]. The problem is small enough to reason about formally but rich enough to expose the classical deadlock conditions catalogued by Coffman, Elphick, and Shoshani [2], as well as liveness, fairness, and distributed coordination issues.

The literature contains deterministic ordering schemes, centralized arbiters, randomized protocols, distributed token/priority methods, self-stabilizing variants, formal-control formulations, and more recently learning-based and multi-agent evaluations [3], [4], [5], [6], [7], [8], [9], [10]. Consequently, a visually appealing schedule is not evidence of novelty. In particular, Lee’s 2023 generalized DPP paper describes a group round-robin strategy in which $\lfloor N/2 \rfloor$ nonadjacent processors execute concurrently and the group shifts right after execution [11]. This substantially anticipates the fixed rotating schedule that motivated the present study.

Rather than rebrand prior art, this paper asks a narrower question: *what can be proved and measured about rotating conflict-free groups, and can the deterministic fairness structure be retained while recovering capacity lost when scheduled philosophers are not hungry?*

The paper is organized around a deliberately asymmetric three-stage progression. Fixed GRR is the formal baseline and replication target; Age-MWIS is a prior-art adaptive reference used to estimate how much performance is available when the deterministic service deadline is relaxed; GF-GRR is the primary extension. The contributions are:

- a graph-theoretic formalization of fixed GRR as a rotation of maximum independent sets of the cycle C_N , together with safety, capacity, and deterministic service-opportunity bounds;
- independent computational verification of the GRR construction for $3 \leq N \leq 500$ and exact cycle independent-set optimization against brute force;
- a unified adaptive reference, Age-MWIS, framed explicitly as a benchmark rather than a novel contribution;
- a guarded-fill extension, GF-GRR, that never removes a hungry philosopher selected by the GRR guard but adds nonconflicting hungry philosophers from otherwise unused capacity;
- a proof that GF-GRR inherits the deterministic GRR service-opportunity bound and weakly dominates GRR in instantaneous service count for any fixed hunger state;

- a paired statistical study quantifying the three-way throughput–latency–bound trade-off, including the fraction of Age-MWIS’s adaptive throughput gain recovered by GF–GRR;
- a corrected native-concurrency implementation using real POSIX threads and fork mutexes, guarded by sanitizer smoke tests and executed as a 960-run paired matrix on native x86₆₄ and ARM64 hardware;
- an explicit comparison of centralized GF–GRR, coordinator-free DGuard, Age-MWIS, a strict arrival-ordered FIFO waiter, and the ordinary $N - 1$ waiter across sleep, ledger, and two-shard key/value payloads under saturated and bursty demand;
- a final semantically corrected native result: completion-window accounting and warmup-consistent DGuard epochs show that coordinator removal is not universally faster. DGuard loses to GF–GRR on saturated sleep on both architectures; on ledger/KV it wins strongly on ARM64 but not on x86₆₄, while producing larger p99 wait and lower fairness in both architectures; and
- a separate process-isolated replication of DGuard using process-shared synchronization, retained as descriptive evidence because it is implemented independently of the corrected thread harness.

We deliberately avoid claiming that either age-aware independent-set scheduling or guarded residual filling is globally unprecedented. Persistent-task fairness, response-time guarantees, age/priority mechanisms, and conflict-graph independent-set scheduling have substantial prior literatures [12], [13], [14], [15]. The claim is narrower: an explicit composition of the Lee-style GRR service guard with work-conserving residual independent-set filling, together with formal analysis and a reproducible empirical characterization under the stated model.

II. LITERATURE REVIEW

A. Review method and scope

We conducted a targeted literature review through 15 September 2026, prioritizing primary publications, publisher/DOI records, archival manuscripts, and monographs. Query families combined *dining philosophers* with *deadlock*, *starvation*, *fairness*, *round robin*, *group scheduling*, *independent set*, *maximum-weight independent set*, *self-stabilizing*, *actor monitor*, *feedback control*, and *multi-agent coordination*. We also followed backward references from foundational papers and searched specifically for formulations that schedule multiple nonadjacent philosophers simultaneously. The goal was not a bibliometric systematic review; it was a novelty- and mechanism-oriented review sufficiently broad to identify direct prior art and the principal algorithmic families against which the present construction should be interpreted.

The review produced four relevant lines of work: (i) deadlock avoidance through ordering, arbitration, or symmetry breaking; (ii) randomized and distributed progress protocols; (iii) fairness- and age-aware schedulers; and (iv) conflict-graph/independent-set scheduling. Most importantly, Lee’s 2023 group round-robin construction directly anticipates the fixed rotating schedule

studied here [11]. This changed the research question from “is the rotation new?” to “what guarantees does the rotation have, where does it waste capacity, and can a conservative adaptive fill preserve its hard service guard?”

B. Origins, deadlock, and structured concurrency

Dijkstra introduced the dining-philosophers paradigm while developing hierarchical disciplines for sequential processes [1]. The DPP directly instantiates mutual exclusion, hold-and-wait, no-preemption, and circular-wait phenomena associated with system deadlocks [2]. Hoare’s monitor abstraction provided a structured shared-memory synchronization framework [16], and his later Communicating Sequential Processes work used DPP-like examples to demonstrate structured communication and synchronization [3].

Simple remedies break one of the deadlock-enabling conditions: impose a resource hierarchy, restrict the number of contenders, acquire resources atomically through an arbiter, or break symmetry. These approaches differ sharply in fairness and scalability. A resource hierarchy can eliminate circular wait but does not by itself guarantee starvation freedom; centralized waiter mechanisms can enforce queue fairness but introduce a coordination bottleneck.

C. Randomized and distributed solutions

Lehmann and Rabin showed the significance of randomization for symmetric, fully distributed dining philosophers, achieving eventual progress with probability one under their assumptions [4]. Dufлот, Fribourg, and Picaronny later modified the randomized construction to remove a fairness assumption on the scheduler and analyzed convergence time [17]. Chandy and Misra generalized resource conflict through the drinking-philosophers problem, using distributed acyclic precedence information to resolve conflicts fairly [5]. Rana and Banerji studied distributed optimality in terms of communication/computation overhead and concurrency [6]; Vaughan examined the fairness/concurrency trade-off in centralized and decentralized forms [18]. Lynch provides a systematic treatment of deterministic and randomized dining-philosopher algorithms in distributed computation [19].

D. Fairness, bounded waiting, and adaptive scheduling

Dijkstra’s later starvation-free exclusion work explicitly treats the generalized graph formulation in which adjacent vertices cannot enter their critical section together [20]. Wedde developed a starvation-free interaction-system solution [21]; Awerbuch and Saks studied polynomial response time for dynamically created, conflicting jobs in a generalized dining-philosophers setting [12]; and Bar-Noy *et al.* formalized fair repeated service for persistent dependent tasks represented by conflict graphs [13]. These works are especially relevant to the present emphasis on response regularity rather than deadlock freedom alone. Self-stabilizing DPP algorithms have also been evaluated on latency, throughput, and fault tolerance [7], while modern randomized locking work studies strong time/fairness guarantees under adversarial delay [22]. Parallel

Actor Monitors provide a relevant precedent for age-ordered reservation-aware scheduling in a dining-philosophers example [14].

Independent-set scheduling is standard outside DPP as well. Wireless-network schedulers routinely construct a conflict graph and choose a maximum-weight independent set (MWIS), often using queue backlog or related urgency measures to trade throughput and delay [15]. This broader literature is why Age-MWIS is used here as an adaptive reference rather than claimed as a contribution. It also motivates the central design question of the merged study: how much of a work-conserving independent-set scheduler's gain can be recovered while retaining a deterministic GRR service guard?

E. Recent reformulations and learning-based studies

Choppella *et al.* recast generalized dining philosophers as feedback control, separating architecture from algorithmic details [8]. In 2026, Özkurt *et al.* compared reinforcement-learning policies with classical methods at up to 160 philosophers [9]. DPBench used DPP as a simultaneous-coordination benchmark for LLM agents and reported deadlock rates above 95% in some simultaneous settings, emphasizing the value of explicit external coordination [10].

F. Direct prior art for rotating groups

Lee's 2023 method is the closest prior art to the fixed schedule studied here: form a group of $\lfloor N/2 \rfloor$ nonadjacent processors, execute them simultaneously, and shift the group right in a round-robin manner [11]. The present paper therefore treats fixed GRR as prior art and a replication target. The research gap we study is dynamic utilization: a fixed group can contain philosophers that are not currently hungry, leaving conflict-free service opportunities unused.

III. MODEL AND DEFINITIONS

Let $C_N = (V, E)$ be the cycle graph with $V = \{0, \dots, N-1\}$ and $E = \{(i, (i+1) \bmod N)\}$. Vertex i is philosopher P_i . A set $S \subseteq V$ is feasible in a round iff it is an independent set of C_N ; then no fork is shared by two simultaneous eaters. The maximum possible number of simultaneous eaters is

$$\alpha(C_N) = \left\lfloor \frac{N}{2} \right\rfloor. \quad (1)$$

Time is discretized into rounds. A philosopher is either thinking or has one pending hunger request. Eating takes one round. If a request is served in its arrival round its waiting age is zero; otherwise age increments by one after each unserved round. This synchronous centralized abstraction intentionally isolates scheduling behavior from OS-thread timing and message-passing effects.

For stochastic experiments, a non-pending philosopher receives a Bernoulli hunger opportunity with probability p_i each round. We evaluate uniform $p_i = p$ and hotspot demand, where approximately 10% of vertices receive four times the mean rate (capped at 0.95) and the remaining rate is adjusted to preserve the requested mean.

IV. FIXED GROUP ROUND-ROBIN

Let $m = \lfloor N/2 \rfloor$. Define the GRR set in round t as

$$S_t = \{(t + 2k) \bmod N : k = 0, \dots, m-1\}. \quad (2)$$

For even N , this alternates the two parity classes. For odd N , the pairwise-spaced set rotates around the odd cycle.

Proposition 1 (Safety and maximal concurrency). For every $N \geq 3$ and every round t , S_t is an independent set of C_N and $|S_t| = \lfloor N/2 \rfloor = \alpha(C_N)$.

Proof. Write $m = \lfloor N/2 \rfloor$. Distinct consecutive elements of the construction differ by two modulo N , so they cannot be adjacent. For even $N = 2m$, the selected vertices form one parity class and the cyclic wrap-around also has distance two. For odd $N = 2m+1$, the selected sequence ends at $t+2(m-1)$; the shorter cyclic distance between this vertex and t is three, not one. Hence S_t is independent. Its cardinality is m . Any independent set on a cycle uses at most one endpoint of each of the m disjoint edges obtained by deleting one vertex (odd case) or pairing consecutive vertices (even case), so $\alpha(C_N) \leq m$. Therefore $|S_t| = \alpha(C_N) = m$. $\square$

Proposition 2 (Service-opportunity gap). Under the rotation above, the gap between consecutive GRR opportunities for a philosopher is exactly two rounds when N is even, and belongs to $\{2, 3\}$ when N is odd.

Proof. Philosopher i is scheduled in round t exactly when $i \equiv t + 2k \pmod{N}$ for some $k \in \{0, \dots, m-1\}$. For even $N = 2m$, $S_{t+2} = S_t$, while S_{t+1} is the complementary parity class; thus each vertex occurs every two rounds. For odd $N = 2m+1$, S_{t+2} is S_t shifted two positions, so most vertices reappear after two rounds. Because m selected positions cannot tile an odd cycle with alternating selected/unselected vertices, one unselected pair occurs in each rotated pattern; crossing this unique two-vertex gap delays the next opportunity by one additional round. Hence the only cyclic inter-opportunity gaps are two and three. $\square$

The construction was exhaustively checked in software for every $3 \leq N \leq 500$: no adjacency/cardinality failures occurred, and the maximum observed inter-opportunity gap was three.

Corollary 1 (Bounded waiting in the round model). If a hunger request remains pending until served, fixed GRR offers service after at most one intervening round for even N and at most two intervening rounds for odd N . This bound assumes unit service time and synchronous round boundaries; it is a scheduler bound, not a claim about arbitrary asynchronous implementations.

The limitation is immediate: GRR reserves positions without inspecting current demand. If a scheduled philosopher is not hungry, the slot remains unused even when another nonadjacent hungry philosopher could eat.

V. ADAPTIVE REFERENCE: AGE-MWIS

Fixed GRR is demand-oblivious: it can leave capacity unused when a scheduled philosopher is not hungry. To separate the value of adaptivity from the value of a deterministic deadline, we use an age-aware maximum-weight independent-set policy as a fully adaptive reference. Let H_t be the hungry set and

TABLE I
POSITIONING RELATIVE TO REPRESENTATIVE DINING PHILOSOPHERS AND CONFLICT-SCHEDULING LITERATURE

Work	Coordination model	Main objective	Fairness/progress mechanism	Relation to this paper
Dijkstra (1971) [1]	Shared-resource processes	Deadlock-safe ordering	Hierarchical resource discipline	Foundational DPP/resource-ordering context; not a round scheduler.
Lehmann–Rabin (1981) [4]	Symmetric distributed	Probabilistic progress	Randomized choices	Solves symmetry without centralized schedules; different assumptions.
Chandy–Misra (1984) [5]	Distributed conflict graph	Fair resource conflict resolution	Dynamic precedence / resource state	General graph-oriented distributed solution; richer message semantics than our round model.
Duflot et al. (2004) [17]	Randomized distributed	Progress without scheduler fairness assumption	Probabilistic convergence	Addresses adversarial scheduling assumptions absent from our centralized model.
Scholliers et al. (2014) [14]	Actor monitor / scheduler	Parallelism with controlled conflicts	Age/order and reservation mechanisms	Important precedent for age-aware reservation-style coordination.
Paschalidis et al. (2015) [15]	Conflict-graph scheduling	Throughput / delay	MWIS with queue-related weights	Establishes independent-set scheduling as a general scheduling pattern.
Lee (2023) [11]	Centralized group round robin	Deadlock prevention with concurrency	Rotate $\lfloor N/2 \rfloor$ non-adjacent processors	Direct prior art for fixed GRR; replicated and formalized here.
Ben-David–Blleloch (2025) [22]	Concurrent locks	Fast fair wait-free locking	Randomization with fairness guarantees	Modern fairness benchmark under different asynchronous lock semantics.
Hasan–BusiReddyGari (2026) [10]	LLM multi-agent contention	Simultaneous coordination	Benchmark, not scheduler	Motivates external conflict-set coordination as a contemporary application.

$a_i(t)$ the current waiting age of hungry philosopher i . Assign each eligible vertex the lexicographic weight

$$q_i(t) = (1, a_i(t)), \quad (3)$$

and choose

$$A_t \in \arg \max_{A \in \mathcal{I}(C_N), A \subseteq H_t}^{\text{lex}} \left(|A|, \sum_{i \in A} a_i(t) \right). \quad (4)$$

Thus Age-MWIS first maximizes the number served in the current round and, among maximum-cardinality choices, prefers the greatest accumulated waiting age. On a cycle this can be solved exactly in $O(N)$ time by two path dynamic programs. The policy is work-conserving with respect to the current hunger state, but it reserves no future deadline for any individual request. Conflict-graph MWIS scheduling is established prior art [13], [15]; Age-MWIS is therefore a benchmark approximating an aggressive adaptive ceiling in this round model, not a co-equal algorithmic contribution.

VI. PRIMARY EXTENSION: GUARDED-FILL GRR

GF–GRR preserves GRR as a hard fairness guard and uses adaptive selection only for residual capacity. Let H_t denote the set of hungry philosophers. The mandatory set is

$$M_t = H_t \cap S_t. \quad (5)$$

Every vertex in M_t is served. Remove M_t , its neighbors, and all non-hungry vertices from consideration. On the remaining induced graph, choose an independent set that lexicographically maximizes (i) number of additionally served requests and then (ii) total waiting age. Because the residual graph is a subgraph of a cycle (hence a union of paths, or a cycle if $M_t = \emptyset$), this optimization is solvable by dynamic programming in $O(N)$ time.

Algorithm 1 Guarded-Fill GRR for round t

- 1: $S_t \leftarrow \{(t + 2k) \bmod N\}_{k=0}^{\lfloor N/2 \rfloor - 1}$
- 2: $M_t \leftarrow H_t \cap S_t$ ▷ mandatory GRR guard
- 3: block M_t and every neighbor of M_t
- 4: $R_t \leftarrow$ hungry, unblocked vertices
- 5: $A_t \leftarrow$ lexicographic max-cardinality/age independent set of R_t
- 6: **return** $M_t \cup A_t$

A. Exact residual optimization on the cycle

For an eligible hungry vertex i with waiting age a_i , assign the lexicographic value $q_i = (1, a_i)$; an ineligible vertex contributes no selectable value. On a path with vertices $1, \dots, L$, let D_j denote the best pair (served-count, summed-age) on the prefix through j . With lexicographic maximum $\max_{\text{lex}}$,

$$D_j = \max_{\text{lex}} (D_{j-1}, D_{j-2} + q_j), \quad (6)$$

whenever j is eligible, and $D_j = D_{j-1}$ otherwise. A cycle is solved by taking the better of two cases: exclude vertex 0, or include vertex 0 and exclude its two neighbors. Backtracking recovers the set. This is $O(N)$ time and $O(N)$ auxiliary memory in the reference implementation (reducible to $O(1)$ value memory if reconstruction is not required). We verified the returned lexicographic objective against brute-force enumeration on 1,500 random cycles with $3 \leq N \leq 12$ and observed zero mismatches.

Proposition 3 (Safety). GF–GRR never schedules adjacent philosophers.

Reason. M_t is independent by Proposition 1; all neighbors of M_t are excluded from A_t ; and A_t is itself independent. $\square$

Proposition 4 (Inherited waiting bound). Every pending request that would be served by fixed GRR at round t is also served by GF–GRR at round t . Therefore GF–GRR inherits Corollary 1’s deterministic opportunity/wait bound.

Proposition 5 (Per-state dominance over GRR). For any fixed H_t , $|\text{GF-GRR}(H_t, t)| \geq |H_t \cap S_t|$. Equality holds only when no additional feasible hungry vertex exists or the residual optimum is empty.

Proposition 5 is deliberately local: because different policies create different future states, it does not by itself prove higher long-run throughput. That question is empirical under the stochastic model.

VII. ALGORITHMIC PROGRESSION AND SECONDARY COMPARATORS

The central comparison is the progression

$$\begin{aligned} \text{fixed GRR} &\longrightarrow \text{Age-MWIS reference} \\ &\longrightarrow \text{GF-GRR hybrid.} \end{aligned} \quad (7)$$

GRR supplies the deterministic bound; Age-MWIS shows what a fully adaptive exact cycle scheduler can gain when that bound is not enforced; GF-GRR asks how much of that gain can be retained while preserving the guard. Two secondary comparators provide additional context:

- 1) **RotatingGreedy**: scan vertices from a rotating start and greedily accept hungry nonadjacent vertices. It is $O(N)$ and work-conserving in many states, but it has no explicit age objective or deterministic request deadline.
- 2) **SingleWaiter**: serve only the oldest pending request, representing a strongly serialized fair baseline.

VIII. EXPERIMENTAL METHODOLOGY

A. Core statistical design

The core benchmark uses $N \in \{101, 1001\}$, $p \in \{0.05, 0.20, 0.40\}$, uniform and hotspot workloads, five algorithms, and 12 paired random seeds per condition. Each run contains 300 warm-up rounds followed by 3000 measured rounds. This is $2 \times 3 \times 2 \times 5 \times 12 = 720$ algorithm runs. Each algorithm receives the same seed within a condition; one random variate is generated for every philosopher in every round so paired streams are structurally aligned even when policy states diverge.

The primary planned comparison is GuardedFill versus GRR on throughput and mean wait (24 tests). Two explicitly secondary contrast families support the merged narrative: Age-MWIS versus GRR on throughput and mean wait (24 tests), and GuardedFill versus Age-MWIS on throughput and maximum observed wait (24 tests). Within each family we apply two-sided Wilcoxon signed-rank tests with Holm correction and report paired mean differences; 95% paired t confidence intervals are retained as effect-size summaries, not as the sole significance criterion. All 24 primary GuardedFill-GRR tests and all 24 Age-MWIS-GRR reference tests remain significant after Holm correction ($p_{\text{Holm}} = 0.011719$ in each condition). For GuardedFill versus Age-MWIS, 20 of 24 contrasts remain significant after correction; the four nonsignificant contrasts occur at the lowest uniform demand ($p = 0.05$), where the two adaptive policies are nearly indistinguishable in throughput and observed maximum wait.

B. Additional regimes

Canonical $N = 5$ experiments use 30 replications and 10,000 measured rounds for intuition at the original problem size. Stress tests use $N = 10,001$, $p \in \{0.2, 0.4\}$, both workload types, five paired replications, and 1200 measured rounds after 200 warm-up rounds. Selector microbenchmarks materialize schedules for $N = 10^3, 10^4, 10^5, 10^6$.

C. Metrics

We record completed meals/round (throughput), normalized utilization relative to $\lfloor N/2 \rfloor$, mean waiting age, 95th/99th percentile wait, maximum observed wait, Jain’s fairness index over per-philosopher completion counts, and implementation time. Maximum observed waiting in a finite stochastic run is not a proof of starvation freedom; the formal bound for GRR/GuardedFill comes from Propositions 2 and 4.

D. Correctness and implementation verification

The GRR construction was checked for adjacency, cardinality, and opportunity gaps for $3 \leq N \leq 500$. The exact cycle Age-MWIS dynamic program was cross-checked against brute-force enumeration on 1,500 random instances for $3 \leq N \leq 12$, optimizing the lexicographic pair (cardinality, total waiting age); there were zero mismatches. Code was implemented in Python/NumPy with Numba JIT. Runtime numbers are machine- and implementation-specific and are used only to assess scaling trends.

E. Final native multi-architecture validation

The native benchmark is a C++17/POSIX-thread implementation with one worker thread per philosopher, real mutex-protected forks, a central scheduler for GF-GRR and Age-MWIS, a coordinator-free DGuard policy, an explicit arrival-ordered head-of-line FIFO waiter, and the ordinary $N - 1$ room-semaphore waiter. Three critical-section payloads are used. The *sleep* payload samples a bounded log-normal-like duration clipped to 0.4–2.6 ms and sleeps while holding the two resource mutexes. The *ledger* payload performs variable arithmetic plus a double-entry transfer across the locked shards; the global balance sum is checked after each run. The *KV* payload performs variable-count hash-like mutations over two 64-entry shard arrays. In the saturated workload a worker immediately requests another critical section after completion; in the bursty workload thinking intervals are 0.6–5.0 ms with a 15% probability of a longer pause. Each worker has at most one outstanding request.

The final native evidence was produced only after two rounds of pre-submission audit. The first audit corrected a recursive control-flow error in the sleep payload and replaced the central final-drain sleep with the selected application payload. The second audit corrected measurement semantics. First, an operation is now counted only if its *completion timestamp*, taken after payload execution and resource release, lies in $[t_{\text{start}}, t_{\text{end}})$. Thus completions/s, Jain fairness, and zero-service counts refer to service actually completed during the measurement interval; requests that arrive before the window

may count if they complete inside it, and requests that complete after t_{end} do not. Second, DGuard uses a dedicated monotonic `epoch_origin_ns` established immediately before worker release, so the rotating epoch evolves during warmup instead of remaining at phase zero until measurement begins. The post-stop drain remains only for clean shutdown and cannot contaminate measured counts because completions after t_{end} are excluded.

The final matrix uses $N \in \{101, 501\}$, payloads {sleep, ledger, KV}, saturated and bursty demand, eight paired seeds, and five policies {GF-GRR, DGuard, FIFO waiter, ordinary waiter, Age-MWIS}. Each run measures 2.0 s after warmup, with a 200 μs scheduler tick, up to 120 μs scheduler jitter, up to 80 μs worker-dispatch jitter, and a 1.2 ms DGuard epoch. This yields

$$\begin{aligned} & 2 \text{ architectures} \times 2N \times 3 \text{ payloads} \times 2 \text{ workloads} \\ & \times 5 \text{ policies} \times 8 \text{ seeds} = 960. \end{aligned}$$

accepted runs.

The x86₆₄ runner exposed two logical CPUs of an AMD EPYC 7763 host; the ARM64 runner exposed two ARM Neoverse-N2 cores. Both used Ubuntu 24.04 GitHub-hosted runner images, Linux 6.17 Azure kernels, and GCC 13.3.0. The ARM64 job reported native `aarch64`; it was not cross-compiled or emulated. Because the machines differ in microarchitecture and virtualization environment, absolute x86₆₄–ARM64 throughput differences are descriptive rather than an ISA comparison.

The accepted source was reconstructed from the previously audited base inside each runner, passed explicit semantic source checks, compiled at `-O0` with AddressSanitizer and UndefinedBehaviorSanitizer for a sleep-payload smoke test, and then compiled with `-O3`. Both runner artifacts contain byte-identical source with SHA-256 `62e91da4636dc597d43e64a6f9a81b5bc2429f2796e497831047779c3476c336`.

We record completions/s, request-to-lock-acquisition mean/p95/p99/max wait, Jain fairness over per-worker completions, minimum/maximum completions, and the number of workers with zero measured completions. Zero-service counts are finite-window diagnostics, not proofs of starvation. For paired contrasts, two-sided exact Wilcoxon signed-rank tests are Holm-corrected within each three-metric family (throughput, p99, Jain) for a fixed architecture, payload, workload, and policy contrast.

IX. RESULTS

A. Canonical $N = 5$ behavior

At $N = 5$, fixed GRR preserves its hard waiting bound but can leave substantial capacity unused under skew. At $p = 0.20$ hotspot demand, GRR achieved 0.583 meals/round with mean wait 1.150, while GuardedFill achieved 0.943 meals/round with mean wait 0.120; both had maximum observed wait 2. At $p = 0.40$ uniform demand, throughput increased from 1.424 to 1.620 meals/round while maximum observed wait again remained 2.

The fairness index requires careful interpretation under hotspot demand: equal service counts are not necessarily

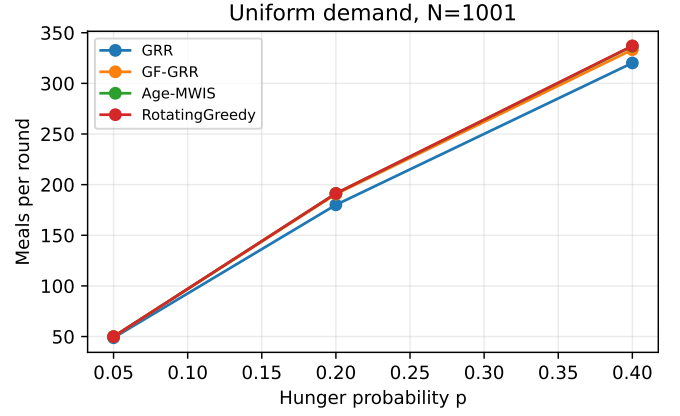


Fig. 1. Throughput at $N = 1001$ under uniform demand. Adaptive schedulers recover slots that fixed GRR leaves idle.

desirable when demand itself is unequal. Thus hotspot Jain values mainly describe equality of completions, not demand-normalized justice.

B. Core throughput and waiting

Figure 1 shows the uniform $N = 1001$ throughput curves. The three-stage progression is visible directly. At $p = 0.20$, GRR completed 180.04 meals/round, Age-MWIS 191.31, and GuardedFill 190.66. Relative to the full GRR-to-Age-MWIS gain, GuardedFill recovered 94.3% while retaining the GRR maximum observed wait of 2 rounds. RotatingGreedy reached 191.33, confirming that the remaining throughput headroom above GuardedFill is small in this condition.

Across uniform $N = 1001$ workloads, GuardedFill throughput gains over GRR were 2.30%, 5.90%, and 4.08% for $p = 0.05, 0.20, 0.40$ respectively; corresponding mean-wait reductions were 89.5%, 55.5%, and 19.5%. Under hotspot demand the throughput gains were 3.23%, 3.52%, and 4.21%, with mean-wait reductions of 77.7%, 43.6%, and 22.1%. More importantly for the merged paper, GF-GRR recovered between 79.3% and 99.7% of the Age-MWIS throughput gain over GRR across all 12 core conditions. Its absolute throughput deficit to Age-MWIS at $N = 1001$ was at most about 1.01%, while its maximum observed wait remained 2 rounds in every condition.

At $N = 1001, p = 0.20$ hotspot demand, GRR throughput was 160.12 and GuardedFill was 165.75 meals/round (paired 95% CI for the difference: 5.602–5.668). Mean wait fell from 0.627 to 0.354 rounds (difference CI: -0.274 to -0.273). Under uniform demand at the same p , the throughput difference was 10.621 meals/round (95% CI 10.597–10.645), and mean wait fell from 0.557 to 0.248.

C. Tail waiting: hard guard versus unconstrained adaptivity

The distinction between GuardedFill and the fully adaptive methods is most visible in tail waiting. In the core runs, GuardedFill and GRR never exceeded a wait of two rounds for odd N , exactly as predicted by the formal guard. Age-MWIS reached maxima between roughly 2 and 7 rounds depending on

TABLE II

THREE-WAY $N = 1001$ TRADE-OFF. VALUES ARE MEANS OVER 12 PAIRED REPLICATIONS. “RECOVERY” IS $(T_{GF} - T_{GRR}) / (T_{Age} - T_{GRR})$: THE FRACTION OF THE AGE-MWIS THROUGHPUT IMPROVEMENT OVER GRR RECOVERED BY GF-GRR. MAX-WAIT COLUMNS REPORT THE LARGEST OBSERVED WAIT ACROSS THE 12 RUNS.

Workload	p	GRR thr.	GF thr.	Age thr.	Recovery	GRR max	GF max	Age max
Uniform	.05	48.806	49.927	49.930	99.7%	2	2	2
Uniform	.20	180.042	190.663	191.307	94.3%	2	2	4
Uniform	.40	320.165	333.239	336.628	79.4%	2	2	6
Hotspot	.05	47.586	49.124	49.190	95.9%	2	2	4
Hotspot	.20	160.119	165.754	165.837	98.5%	2	2	5
Hotspot	.40	303.330	316.107	318.575	83.8%	2	2	5

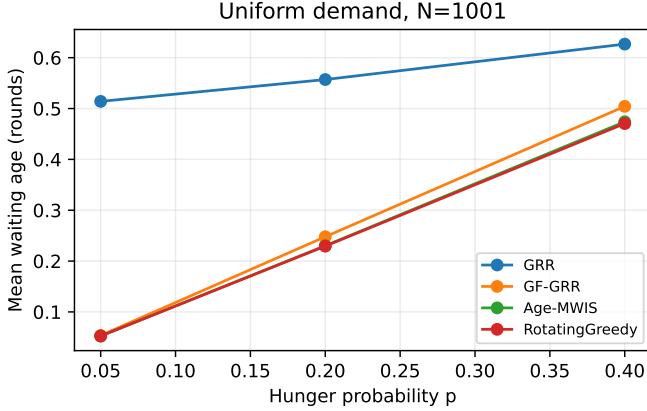


Fig. 2. Mean waiting age at $N = 1001$ under uniform demand. Fixed GRR pays a utilization/latency penalty for deterministic reservation; GuardedFill recovers much of it without relaxing the GRR guard.

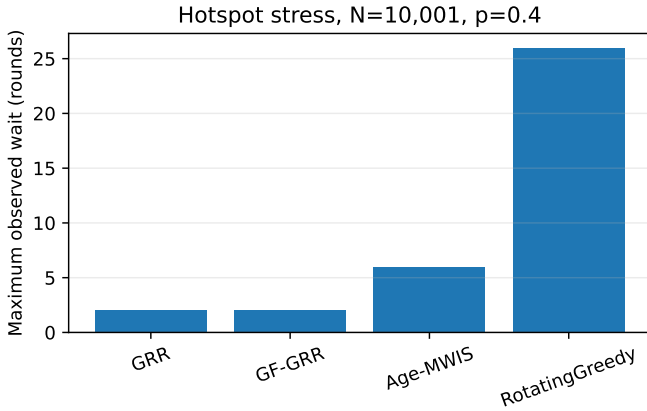


Fig. 3. Maximum observed waiting at $N = 10,001$, hotspot $p = 0.4$. GuardedFill preserves the fixed GRR tail bound in this model, unlike unconstrained adaptive comparators.

condition, while RotatingGreedy produced substantially longer tails under skew (up to 25 rounds in the core benchmark).

At $N = 10,001$, $p = 0.4$ hotspot demand, maximum observed waits were 2 for GRR, 2 for GuardedFill, 6 for Age-MWIS, and 26 for RotatingGreedy (Fig. 3). Throughput in the same condition was 3032.18, 3158.72, 3182.76, and 3184.03 meals/round respectively. Thus GuardedFill gave up about 0.8% throughput relative to the Age-MWIS adaptive reference but retained the deterministic GRR guard.

TABLE III
SELECTED STRESS RESULTS AT $N = 10,001$

Workload	Algorithm	Throughput	Mean wait	Max wait
Uniform $p = .2$	GRR	1799.74	0.556	2
	GuardedFill	1905.07	0.248	2
	Age-MWIS	1911.61	0.230	5
	RotGreedy	1911.84	0.229	9
Hotspot $p = .4$	GRR	3032.18	0.660	2
	GuardedFill	3158.72	0.515	2
	Age-MWIS	3182.76	0.489	6
	RotGreedy	3184.03	0.487	26

D. Large- N stress results

Table III summarizes selected $N = 10,001$ runs. Under uniform $p = 0.2$, GuardedFill improved throughput from 1799.74 to 1905.07 meals/round (5.85%) and reduced mean wait from 0.556 to 0.248, with the same maximum observed wait of 2. Age-MWIS achieved 1911.61 and RotatingGreedy 1911.84 but with maxima of 5 and 9 rounds respectively.

E. Capacity–latency trade-off

The adaptive policies expose a meaningful distinction between *work conservation* and *bounded service*. Age-MWIS and RotatingGreedy use current demand more aggressively and usually obtain the highest throughput, but neither reserves a deadline for any particular request. GuardedFill instead constrains adaptation by forcing the base GRR obligations first. At $N = 10,001$, $p = 0.4$ hotspot demand, for example, GuardedFill delivered 3158.72 meals/round versus 3182.76 for Age-MWIS (a 0.76% gap) while reducing the observed maximum wait from 6 to 2. RotatingGreedy reached 3184.03 but its maximum observed wait rose to 26. The result is not a universal dominance claim; it quantifies a design choice between a hard deterministic guard and a small amount of additional opportunistic capacity.

F. Selector scaling

Materialized selector timing was consistent with linear growth in N in the tested implementation. At $N = 10^6$, median per-call times recorded in the artifact were approximately 1.05 ms for GRR schedule materialization, 5.67 ms for RotatingGreedy, and 6.43 ms for Age-MWIS. These figures are not portable performance claims: Numba compilation, CPU microarchitecture, memory bandwidth, and representation choices

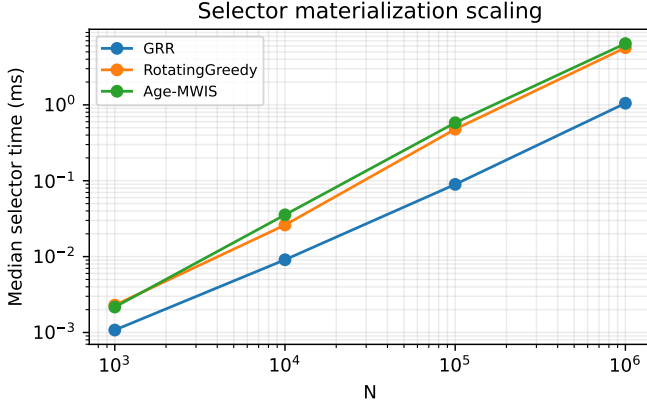


Fig. 4. Selector scaling in the experimental implementation. All shown methods are linear-time on the cycle; fixed GRR has the smallest constant factor.

matter. GF-GRR additionally invokes residual optimization only after constructing its mandatory guard, so its end-to-end simulation cost is reported separately in the raw run records rather than inferred from this selector-only figure. In a production implementation, fixed GRR can also be represented implicitly rather than materializing a million-element mask.

X. FINAL SEMANTICALLY CORRECTED NATIVE MULTI-ARCHITECTURE VALIDATION

A. Saturated sleep: guarded admission controls tails

Table IV summarizes the final saturated sleep results at $N = 501$. The 2.0 s completion-window accounting removes the earlier artifact in which requests that arrived inside a short window could complete long after the window and still count toward throughput/fairness. Under the final semantics, GF-GRR and Age-MWIS both provide service to every worker on every saturated $N = 501$ sleep run; DGuard does as well. The ordinary waiter remains highly concentrated under saturation.

On x86₆₄, GF-GRR averaged 60,539 completions/s, essentially identical to Age-MWIS at 60,941/s, but p99 wait was 17.74 ms versus 204.79 ms and Jain fairness was 0.979 versus 0.954. The GF-GRR/Age-MWIS throughput contrast is not significant after Holm correction ($p_{\text{Holm}} = 0.742$), whereas p99 and Jain are ($p_{\text{Holm}} = 0.0234$). DGuard was slower at 42,382/s and had 78.54 ms p99; GF-GRR’s throughput and p99 advantages both survive Holm correction ($p_{\text{Holm}} = 0.0234$), while their Jain difference does not ($p_{\text{Holm}} = 0.547$).

On ARM64, Age-MWIS obtains the highest sleep throughput (124,348/s), 7.3% above GF-GRR (115,919/s), but GF-GRR reduces p99 wait from 26.04 to 17.05 ms and raises Jain fairness from 0.831 to 0.998. DGuard is both slower than GF-GRR (78,713/s) and higher-tailed (39.05 ms p99). All three GF-GRR/Age-MWIS and GF-GRR/DGuard metrics survive their respective three-test Holm families on ARM64 ($p_{\text{Holm}} = 0.0234$). The important cross-platform observation is therefore not identical throughput rank but the repeated tail-regularity benefit of GF-GRR under the timing-dominated saturated workload.

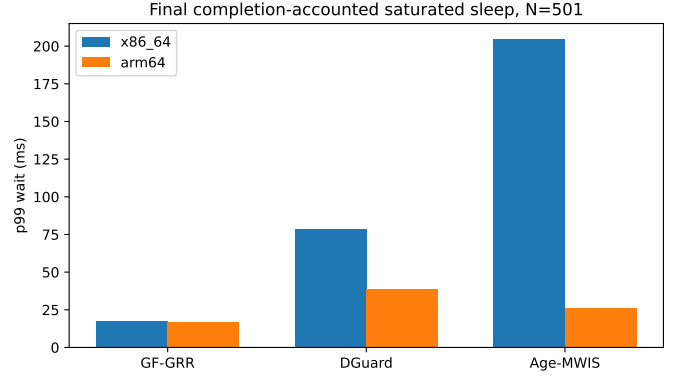


Fig. 5. Final saturated $N = 501$ p99 request-to-lock wait for GF-GRR, DGuard, and Age-MWIS. The qualitative GF-GRR tail advantage persists on both native architectures.

B. Ledger and KV: coordinator-removal throughput is architecture dependent

Table V shows a different operating point for state-mutating critical sections, but the throughput effect of coordinator removal does not replicate uniformly across hardware. On x86₆₄, ledger is essentially tied: DGuard averages 166,603/s versus 166,039/s for GF-GRR (+0.34%), and the paired throughput contrast is not significant after Holm correction ($p_{\text{Holm}} = 0.250$). For KV, GF-GRR is higher at 133,129/s versus 129,002/s for DGuard; that throughput contrast is also not significant ($p_{\text{Holm}} = 0.078$). On ARM64, by contrast, DGuard rises from 177,665 to 275,775/s (+55.2%) for ledger and from 176,109 to 261,439/s (+48.5%) for KV; both throughput differences survive Holm correction ($p_{\text{Holm}} = 0.0234$).

The regularity price is much more consistent than the throughput effect. DGuard p99 wait is larger in all four cases: 13.93 versus 5.95 ms (x86 ledger), 18.99 versus 7.25 ms (x86 KV), 9.13 versus 5.50 ms (ARM ledger), and 9.15 versus 5.55 ms (ARM KV). All four p99 differences survive Holm correction ($p_{\text{Holm}} = 0.0234$). Jain fairness likewise falls from 0.997/0.997 under GF-GRR to 0.963/0.946 on x86₆₄ and from 0.997/0.997 to 0.884/0.910 on ARM64; all four fairness contrasts survive correction. Every accepted ledger run preserves the double-entry conservation invariant.

Age-MWIS sits close to GF-GRR on these application-shaped payloads. GF-GRR has lower p99 than Age-MWIS in all four cases, and every p99 contrast survives Holm correction. Their throughput differences do not survive correction in any of the four architecture/payload comparisons, and the Jain differences are very small. This reinforces that the main distinction between GF-GRR and Age-MWIS in these payloads is tail control rather than a universal throughput ranking.

C. Bursty demand: coordination overhead dominates more often

Under bursty demand all five policies have far more regular completion distributions, and zero-service counts disappear at $N = 501$. On ARM64 sleep, GF-GRR, Age-MWIS, and the ordinary waiter cluster near 77k/s, while DGuard is slightly slower; on x86₆₄ sleep the ordinary waiter is fastest at 71,846/s,

TABLE IV

FINAL SEMANTICALLY CORRECTED SATURATED SLEEP RESULTS AT $N = 501$ (MEANS OVER EIGHT PAIRED SEEDS PER ARCHITECTURE; 2.0 S COMPLETION WINDOW). “ZERO” IS THE NUMBER OF WORKERS WITH NO COMPLETION INSIDE THE MEASUREMENT INTERVAL.

Arch.	Policy	Throughput/s	p99 ms	Jain	Zero
x86 ₆₄	GF-GRR	60538.7	17.735	0.979	0.0
	DGuard	42382.1	78.537	0.962	0.0
	Age-MWIS	60940.7	204.791	0.954	0.0
	FIFO waiter	1145.3	549.198	0.982	0.0
	$N - 1$ waiter	593.6	2016.252	0.103	218.3
ARM64	GF-GRR	115919.2	17.046	0.998	0.0
	DGuard	78712.8	39.053	0.989	0.0
	Age-MWIS	124348.3	26.043	0.831	0.0
	FIFO waiter	1052.8	710.125	0.971	0.0
	$N - 1$ waiter	588.0	1620.006	0.034	389.3

TABLE V

FINAL SATURATED LEDGER/KV RESULTS AT $N = 501$ (MEANS OVER EIGHT PAIRED SEEDS PER ARCHITECTURE; 2.0 S COMPLETION WINDOW).

Arch.	App	Policy	Throughput/s	p99 ms	Jain
x86 ₆₄	Ledger	GF-GRR	166039.2	5.951	0.997
		DGuard	166602.7	13.932	0.963
		Age-MWIS	164640.6	7.673	0.997
	KV	GF-GRR	133129.2	7.249	0.997
		DGuard	129002.1	18.985	0.946
ARM64	Ledger	GF-GRR	177665.4	5.497	0.997
		DGuard	275775.4	9.130	0.884
		Age-MWIS	173640.6	7.076	0.997
	KV	GF-GRR	176108.9	5.547	0.997
		DGuard	261439.3	9.154	0.910
ARM64	Ledger	GF-GRR	174109.7	7.263	0.997
		DGuard	174109.7	7.263	0.997
		Age-MWIS	174109.7	7.263	0.997
	KV	GF-GRR	174109.7	7.263	0.997
		DGuard	174109.7	7.263	0.997

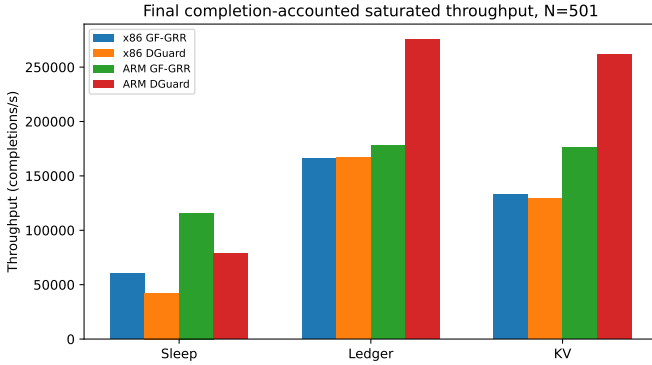


Fig. 6. Final saturated $N = 501$ throughput. DGuard loses on the timing-dominated sleep workload on both architectures; its ledger/KV throughput advantage is strong on ARM64 but does not replicate on x86₆₄. Absolute x86₆₄–ARM64 differences are not interpreted as ISA effects.

compared with 52,739/s for GF-GRR. For ledger/KV, DGuard and the ordinary waiter are fastest on both architectures. For example, x86₆₄ ledger reaches 123,150/s for DGuard and 123,538/s for the waiter versus 76,216/s for GF-GRR; ARM64 ledger reaches roughly 124.6–124.7k/s for DGuard/waiter versus 118.3k/s for GF-GRR. Jain fairness is approximately 0.99–1.00 throughout these bursty $N = 501$ conditions. Thus, when conflicts are intermittent, the global selection pass can cost more than the regularity it provides.

D. What the final native study establishes

The final measurement semantics substantially improve interpretability. Throughput is now literally completions per second inside the measurement interval, and fairness/zero-service counts are based on those same in-window completions. Long post-window drains cannot inflate measured throughput or produce apparent multi-second p99 values from a nominal 180 ms cohort. DGuard also begins rotating before warmup, eliminating the previous initialization asymmetry.

With those corrections, the qualitative result is narrower but stronger. GF-GRR is not a universal throughput winner. Its most reproducible native advantage is controlled p99/tail behavior under sustained contention, especially on the timing-dominated sleep payload. DGuard can be markedly faster for compute/memory payloads on ARM64, but that throughput gain does not reproduce on x86₆₄; what does reproduce is DGuard’s larger p99 and lower Jain fairness relative to GF-GRR for saturated ledger/KV. Age-MWIS can match or exceed GF-GRR throughput, yet GF-GRR retains lower p99 in every saturated $N = 501$ GF-GRR/Age-MWIS comparison. The systems problem is therefore a workload- and platform-dependent throughput–tail–fairness frontier rather than a single winning policy.

XI. COORDINATOR REMOVAL AND PROCESS-ISOLATED REPLICATION

DGuard removes the $O(N)$ central selector. Each worker derives the current rotating guard phase from a shared mono-

tonic epoch, persists an owed flag when it belongs to that phase, refuses opportunistic admission adjacent to an owed neighbor, and attempts to acquire its two mutexes locally. This is coordinator-free shared-memory admission, not a network-distributed protocol: workers still observe a common time base and shared owed state. The corrected thread matrix above shows that this mechanism occupies different operating points depending on both critical-section payload and host platform. It is slower and less regular than GF-GRR on saturated sleep; for ledger/KV it is markedly faster on ARM64 but not on x86₆₄, while giving up p99 regularity and fairness on both.

A. Separate-process replication

To test whether the decentralized mechanism depends on one thread scheduler or one address space, we also implemented a separate fork-per-philosopher program using process-shared POSIX mutexes and semaphores. This process implementation is independent of the affected thread harness: its sleep path directly performs the bounded sampled sleep and does not contain the recursive payload branch. DGuard uses no parent scheduling loop after launch; each child derives guard membership locally from the monotonic epoch and communicates only through process-shared resource mutexes and owed flags. Resource ordering and the ordinary $N - 1$ waiter serve as low-coordination baselines.

The process matrix contains 78 completed runs over $N \in \{5, 51, 201\}$ with sleep and ledger payloads plus $N = 501$ saturated stress cases. All ledger invariants passed. At saturated $N = 201$ with sleep, process DGuard averaged 42,958 completions/s, 19.08 ms p99 wait, Jain 0.976, and zero unserved workers. At $N = 501$, its single stress run reached 55,800/s but p99 rose to 64.76 ms, Jain fell to 0.706, and 28 workers received no measured service. The ledger payload shows the same scale sensitivity: DGuard Jain decreased from 0.964 at $N = 201$ to 0.855 at $N = 501$. Because the process study has only two seeds per core size and one $N = 501$ stress replicate, it remains descriptive replication rather than primary inferential evidence.

The process result is therefore used only to support a limited point: eliminating a central selector does not automatically preserve service regularity as contention and scale rise. It is not used to restore the superseded thread-level throughput claims.

XII. DISCUSSION

The simulator and corrected native experiments now support a more constrained and more defensible interpretation. In the synchronous model, GF-GRR retains most of Age-MWIS’s utilization gain while preserving the formal GRR service guard. In native execution, the strongest replicated property is not universal throughput superiority but service regularity under sustained timing-dominated contention. At saturated $N = 501$ sleep, GF-GRR produces almost identical p99 wait on x86₆₄ and ARM64 despite very different absolute throughput, and it serves every worker in each short measurement window. Age-MWIS trades that regularity for higher throughput, while

the ordinary and strict-FIFO waiters collapse under this high-contention condition.

Coordinator removal is more subtle than the superseded results implied. DGuard does not simply “remove overhead”: with the corrected sleep payload it is slower than GF-GRR on both architectures; on ledger and KV it delivers substantially more throughput on ARM64 but no statistically resolved throughput benefit on x86₆₄. The likely explanation is not an architectural universal but a balance among selection overhead, critical-section cost, memory/arithmetic work, local mutex races, runnable contention, and host scheduling. By contrast, the p99 and fairness penalties of DGuard relative to GF-GRR on saturated ledger/KV reproduce on both platforms. The design question is therefore *how much global coordination is worth paying for at a given contention pattern, payload, platform, and fairness target*, not whether centralization or decentralization is intrinsically superior.

This is not evidence that GRR should replace classical lock protocols. Chandy–Misra, Lehmann–Rabin, resource ordering, fair queueing locks, and distributed/wait-free algorithms solve different problems under different timing, failure, locality, and communication assumptions [4], [5], [22]. Central GF-GRR is best interpreted as an explicit conflict scheduler with a rotating service guard. DGuard shows that the priority idea can be decentralized in shared memory, but local arbitration then becomes part of the problem. A genuinely network-distributed guarded scheduler remains a separate design problem.

That interpretation also connects the result to modern multi-agent systems. DPBench’s observation that independent LLM agents can converge on identical resource-acquisition strategies and deadlock under simultaneous action [10] suggests one possible application domain for external conflict-set schedulers. This paper does not test LLM agents directly, and no performance transfer is assumed.

XIII. THREATS TO VALIDITY AND LIMITATIONS

Correction scope and superseded native data. Two pre-submission audits changed the native evidence. The first fixed a recursive sleep-payload path and a central-drain payload mismatch. The second replaced request-arrival-cohort accounting with completion-window accounting and gave DGuard an epoch origin established before warmup. All thread-level native tables produced before these corrections are superseded and excluded from the final empirical argument. The synchronous simulator is unaffected. The independent process-isolated program is retained only as descriptive evidence and is not pooled with the final 960-run thread matrix.

Hardware replication is not controlled architecture benchmarking. The final matrix runs natively on x86₆₄ and ARM64, but the GitHub-hosted Azure VMs differ in CPU implementation, cache/SMT topology, placement, and virtualization conditions. Absolute throughput differences must not be attributed to ISA. The meaningful cross-platform evidence is replication of qualitative scheduler behavior.

Finite measurement windows. Each final native run measures 2.0 s. This is more than an order of magnitude longer than the superseded short-window study and removes the arrival-cohort/drain artifact, but it is still finite. A zero-service worker

is a windowed diagnostic, not proof of starvation. In the final $N = 501$ matrix GF-GRR, DGuard, Age-MWIS, and FIFO have zero zero-service workers in the headline saturated sleep conditions; the ordinary waiter still shows severe concentration.

Round bound versus wall-clock bound. Proposition 4 applies to the synchronous round model. Variable service duration, preemption, mutex wake-up policy, and scheduler jitter prevent translating a two-round opportunity bound into a fixed millisecond real-time guarantee. Native claims concern measured tail regularity, not a new hard response-time theorem.

Workload model. Demand remains closed-loop with at most one outstanding request per worker. Sleep is a bounded timing surrogate; ledger and KV are application-shaped microbenchmarks rather than production traces. Correlated external arrivals, unbounded heavy tails, I/O stalls, priorities, cancellation, failures, and transactional rollback remain untested.

DGuard is shared-memory decentralized, not network distributed. Workers use a common monotonic time base and shared owed state. Message delay, clock skew, partial failure, partitions, and replicated state are absent. Chandy-Misra-style message-passing comparisons remain future work.

Waiter baselines. The ordinary $N - 1$ waiter inherits host POSIX semaphore/mutex wake-up behavior and has no fairness guarantee. The strict FIFO waiter intentionally stops at the oldest blocked request and can suffer severe head-of-line blocking. More sophisticated fair admission queues with bounded bypass may occupy better operating points.

Statistics. The final native matrix uses eight paired seeds per architecture/payload/workload condition. Seeds align pseudorandom payload sequences, but OS scheduling remains nondeterministic. Exact paired Wilcoxon tests with Holm correction are therefore small-sample robustness checks, not substitutes for dedicated hardware replication. We report nonsignificant contrasts explicitly; for example, GF-GRR and Age-MWIS saturated sleep throughput are statistically indistinguishable on x86₆₄ under the stated family.

Novelty. The fixed rotating schedule is prior art [11]. We did not identify the exact guarded-fill composition in the reviewed sources, but age-priority reservations and MWIS conflict scheduling are established concepts [14], [15]. The contribution is the explicit guarded composition, its formal analysis, and a corrected reproducible characterization under the stated model—not a claim of a universal new DPP solution.

XIV. REPRODUCIBILITY

The accompanying artifact contains the simulator source, raw per-run CSVs, aggregates, structural-verification outputs, selector microbenchmarks, statistical tables, plotting code, manuscript source, and the reproduction notebook. The final native package adds the semantically corrected C++17 source, separate x86₆₄ and ARM64 host metadata, ASan/UBSan smoke outputs, per-architecture 480-row CSVs, the combined 960-row dataset, grouped summaries, paired Wilcoxon/Holm tables, and the figures used in this revision.

The source executed in the final native matrix is byte-identical across runner types with SHA-256 62e91da4636dc597d43e64a6f9a81b5bc2429f2796e49783

1047779c3476c336. The final parallel GitHub Actions execution is run 35013398709; the triggering workflow commit is 6b3ae39566005f637e3518579dce1b9dd297975b. Two native architecture-validation jobs reconstruct the source, require the exact SHA, and pass ASan/UBSan smoke tests. Twenty-four condition jobs then run the same source, one architecture- N -payload-workload condition per native VM, keeping all eight seeds and five policies together within each condition; each condition artifact is required to contain 40 data rows.

Exactly 480 validated rows are required per architecture after merging the 12 condition artifacts, for 960 native rows total. The merged dataset is checked for the full Cartesian product of architectures, N , payloads, workloads, seeds, and policies, with a 2.0 s measurement window in every row. The executable checks the ledger conservation invariant after every ledger run. Earlier affected CSVs are retained only as audit history and are not evidence for any final native claim.

Three additional checks guard the model-level implementation. GRR adjacency, cardinality, and cyclic opportunity gaps were verified for all $N = 3, \dots, 500$ with zero failures. The Age-MWIS cycle dynamic program was compared with exhaustive enumeration on 1,500 random instances for $3 \leq N \leq 12$, with zero objective mismatches. Stress and canonical simulator experiments use separate configurations from the core aggregation.

XV. CONCLUSION

The fixed rotating-group idea that motivated this study is not new: Lee’s 2023 group round-robin construction already captures its core. Formalizing GRR as rotating maximum independent sets nevertheless makes its value precise. In the synchronous model it excludes resource conflicts at admission, attains $\lfloor N/2 \rfloor$ saturated concurrency, and provides service opportunities every two rounds for even N and with gaps of two or three for odd N .

The simulator separates two responses to GRR’s dynamic-utilization loss. Age-MWIS is the aggressive adaptive reference; GF-GRR is the guarded hybrid. Across the simulator’s core conditions GF-GRR recovered 79.3–99.7% of the Age-MWIS throughput improvement over GRR while retaining the formal guard. That establishes a model-level throughput-bound trade-off but says nothing by itself about operating-system scheduling cost.

The final native study was frozen only after correcting both control-flow and measurement-semantics defects. Across 960 two-second runs on native x86₆₄ and ARM64, the strongest replicated GF-GRR property is tail regularity under sustained contention. On saturated sleep, GF-GRR has p99 waits near 17–18 ms on both platforms; DGuard is about 30–32% slower and substantially higher-tailed. Age-MWIS is statistically indistinguishable from GF-GRR in x86₆₄ throughput and is 7.3% faster on ARM64, but GF-GRR retains materially lower p99 and higher fairness on both.

Coordinator removal is valuable only in some regimes. On ARM64 ledger/KV, DGuard raises saturated throughput by 48.5–55.2% relative to GF-GRR, with statistically resolved increases

in p99 wait and decreases in Jain fairness. On x86₆₄ ledger/KV, the DGuard/GF–GRR throughput contrasts are not significant, while the p99 and fairness penalties remain significant. Under bursty demand, low-coordination DGuard/waiter mechanisms are often fastest and fairness is high across policies. The resulting design space is therefore not “centralized good” or “decentralized good.” It is a workload- and platform-dependent frontier among useful work, global selection overhead, local contention, tail latency, and fairness.

The separate process-isolated study remains descriptive evidence that the coordinator-free mechanism can be implemented outside one thread address space, but it is not pooled with the final thread matrix. A genuinely distributed guarded scheduler would need message passing, clock-skew tolerance, failure semantics, and replicated state. The next useful validation is therefore controlled bare-metal replication and then network-distributed coordination, not another claim of a universal Dining Philosophers solution.

REFERENCES

- [1] E. W. Dijkstra, “Hierarchical ordering of sequential processes,” *Acta Informatica*, vol. 1, no. 2, pp. 115–138, 1971.
- [2] E. G. Coffman, Jr., M. J. Elphick, and A. Shoshani, “System deadlocks,” *ACM Computing Surveys*, vol. 3, no. 2, pp. 67–78, 1971.
- [3] C. A. R. Hoare, “Communicating sequential processes,” *Communications of the ACM*, vol. 21, no. 8, pp. 666–677, 1978.
- [4] D. Lehmann and M. O. Rabin, “On the advantages of free choice: A symmetric and fully distributed solution to the dining philosophers problem,” in *Proceedings of the 8th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, 1981, pp. 133–138.
- [5] K. M. Chandy and J. Misra, “The drinking philosophers problem,” *ACM Transactions on Programming Languages and Systems*, vol. 6, no. 4, pp. 632–646, 1984.
- [6] S. P. Rana and D. K. Banerji, “An optimal distributed solution to the dining philosophers problem,” *International Journal of Parallel Programming*, vol. 15, no. 4, pp. 327–335, 1986.
- [7] J. Adamek, G. Farina, M. Nesterenko, and S. Tixeuil, “Evaluating and optimizing stabilizing dining philosophers,” *Journal of Parallel and Distributed Computing*, vol. 109, pp. 63–74, 2017.
- [8] V. Choppella, A. Sanjeev, K. Viswanath, and B. Jayaraman, “Generalised dining philosophers as feedback control,” in *Distributed Computing and Internet Technology (ICDCIT 2020)*, ser. Lecture Notes in Computer Science, 2020, pp. 144–164.
- [9] C. Özkurt, A. Kala, and M. E. Onay, “A reinforcement learning approach to the dining philosophers problem,” *Konya Journal of Engineering Sciences*, vol. 14, no. 2, pp. 613–627, 2026.
- [10] N. Hasan and P. BusiReddyGari, “DPBench: Structural determinants of multi-agent llm coordination under simultaneous resource contention,” arXiv preprint arXiv:2602.13255v2 [cs.AI], Jun. 2026, version 2, revised June 3, 2026; 20 pages, 4 figures; cross-listed in Multiagent Systems (cs.MA); DOI: 10.48550/arXiv.2602.13255. [Online]. Available: <https://arxiv.org/abs/2602.13255>
- [11] S.-U. Lee, “Algorithm for deadlock prevention of generalized philosophers’ dining problem,” *The Journal of Intelligent Information*, vol. 23, no. 2, pp. 73–78, 2023.
- [12] B. Awerbuch and M. E. Saks, “A dining philosophers algorithm with polynomial response time,” in *Proceedings of the 31st Annual Symposium on Foundations of Computer Science (FOCS)*, 1990, pp. 65–74.
- [13] A. Bar-Noy, A. Mayer, B. Schieber, and M. Sudan, “Guaranteeing fair service to persistent dependent tasks,” *SIAM Journal on Computing*, vol. 27, no. 4, pp. 1168–1189, 1998.
- [14] C. Scholliers, E. Tanter, and W. D. Meuter, “Parallel actor monitors: Disentangling task-level parallelism from data partitioning in the actor model,” *Science of Computer Programming*, vol. 80, pp. 52–64, 2014.
- [15] I. C. Paschalidis, F. Huang, and W. Lai, “A message-passing algorithm for wireless network scheduling,” *IEEE/ACM Transactions on Networking*, vol. 23, no. 5, pp. 1528–1541, 2015.
- [16] C. A. R. Hoare, “Monitors: An operating system structuring concept,” *Communications of the ACM*, vol. 17, no. 10, pp. 549–557, 1974.
- [17] M. Duffot, L. Fribourg, and C. Picaronny, “Randomized dining philosophers without fairness assumption,” *Distributed Computing*, vol. 17, no. 1, pp. 65–76, 2004.
- [18] J. G. Vaughan, “The dining philosophers problem and its decentralisation,” *Microprocessing and Microprogramming*, vol. 35, no. 1–5, pp. 455–462, 1992.
- [19] N. A. Lynch, *Distributed Algorithms*. Morgan Kaufmann, 1996.
- [20] E. W. Dijkstra, “Two starvation-free solutions of a general exclusion problem,” 1977, eWD625, E. W. Dijkstra Archive.
- [21] H. F. Wedde, “A starvation-free solution of the dining philosophers’ problem by use of interaction systems,” in *Mathematical Foundations of Computer Science 1981*, ser. Lecture Notes in Computer Science, vol. 118, 1981, pp. 534–543.
- [22] N. Ben-David and G. E. Blelloch, “Fast and fair randomized wait-free locks,” *Distributed Computing*, vol. 38, pp. 51–72, 2025.